

Progress Report

Day 5 — Schema Design & Authentication Structure

Task Management MERN Stack Platform

Shafin Nigamana | 23 May 2026

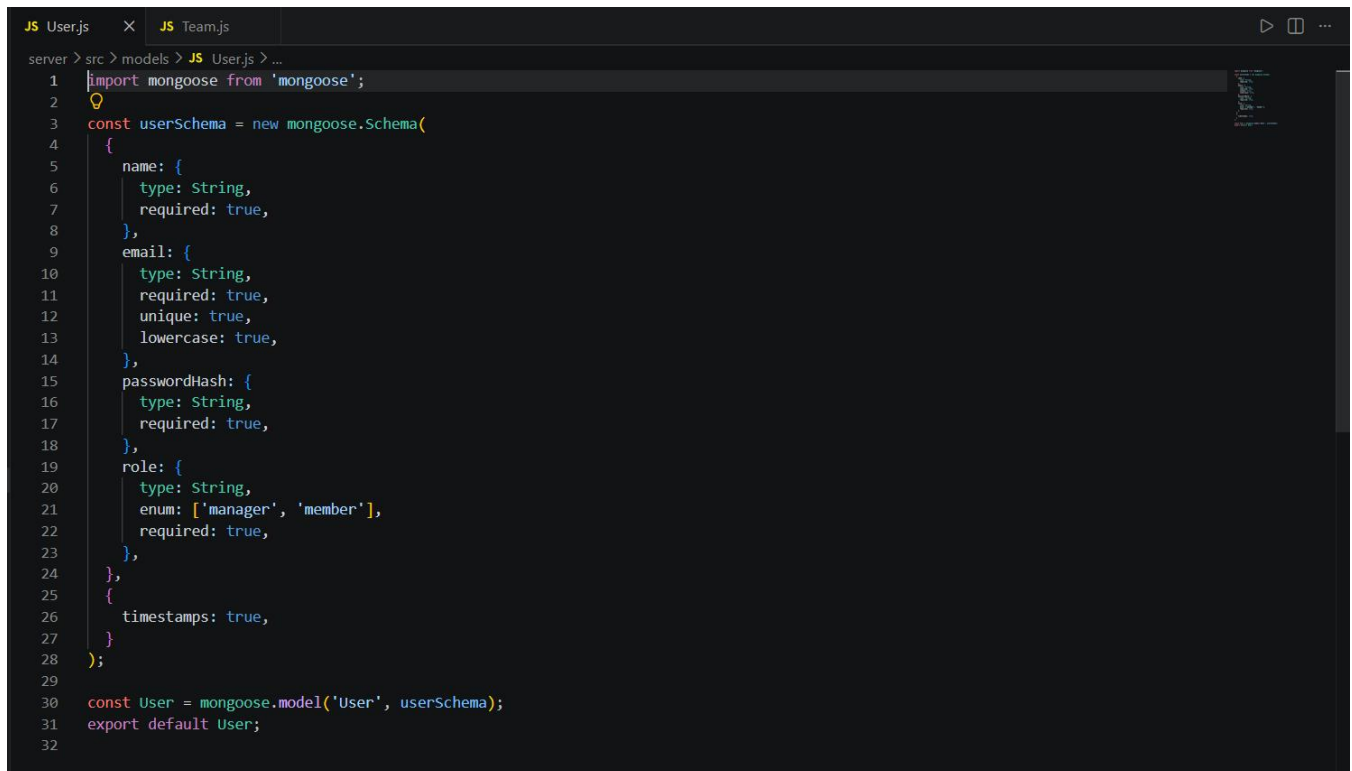
1. Day-5 Objective

The goal for Day 5 was to begin backend application architecture by creating Mongoose schemas for User and Team models and preparing the authentication folder structure for upcoming JWT implementation.

2. Tasks Completed

User Schema Creation

- Created User.js model inside server/src/models

A screenshot of a code editor with a dark theme. The editor shows a file named 'User.js' with the following code:

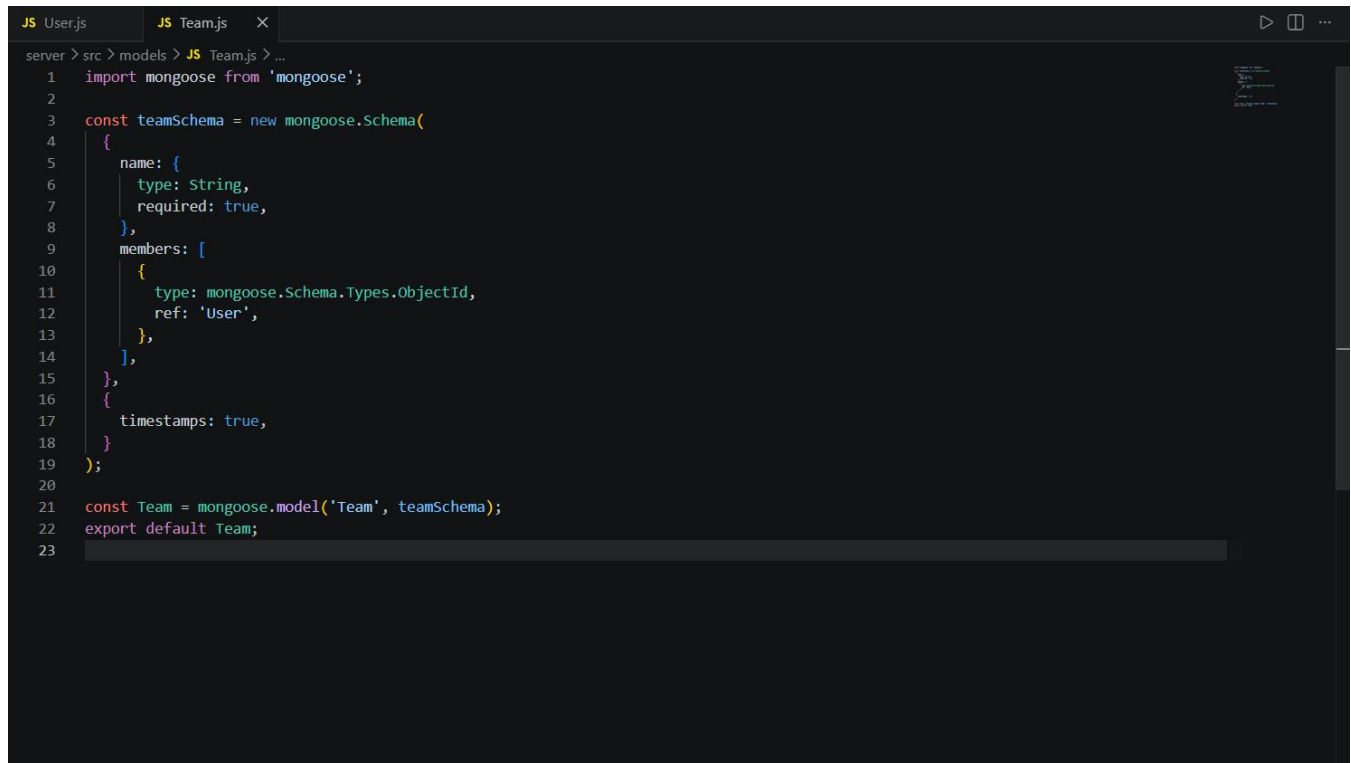
```
1 import mongoose from 'mongoose';
2
3 const userSchema = new mongoose.Schema(
4   {
5     name: {
6       type: String,
7       required: true,
8     },
9     email: {
10      type: String,
11      required: true,
12      unique: true,
13      lowercase: true,
14    },
15     passwordHash: {
16      type: String,
17      required: true,
18    },
19     role: {
20      type: String,
21      enum: ['manager', 'member'],
22      required: true,
23    },
24   },
25   {
26     timestamps: true,
27   }
28 );
29
30 const User = mongoose.model('User', userSchema);
31 export default User;
32
```

The code is written in JavaScript and uses Mongoose to define a schema for a 'User' model. The schema includes fields for name, email, passwordHash, and role, with various validation rules like required, unique, and lowercase. It also includes timestamps. The model is named 'User' and is exported as the default export.

Figure 2.1 — User schema model created using Mongoose.

Team Schema Creation

- Created Team.js model inside server/src/models
- Added team name field and members array
- Configured ObjectId references for User relationships using Mongoose refs

A screenshot of a code editor with a dark theme. The editor shows a file named 'Team.js' with the following code:

```
1 import mongoose from 'mongoose';
2
3 const teamSchema = new mongoose.Schema(
4   {
5     name: {
6       type: String,
7       required: true,
8     },
9     members: [
10      {
11        type: mongoose.Schema.Types.ObjectId,
12        ref: 'User',
13      },
14    ],
15  },
16  {
17    timestamps: true,
18  }
19 );
20
21 const Team = mongoose.model('Team', teamSchema);
22 export default Team;
```

The code defines a Mongoose schema for a 'Team' model. It includes a 'name' field of type 'String' and a 'members' array of 'ObjectId' types, each with a 'ref' to the 'User' model. The schema is then used to create the 'Team' model and is exported as the default.

Figure 2.2 — Team schema with User reference relationship.

Authentication Structure Setup

- Created backend authentication structure:
 - controllers/authController.js
 - routes/auth.js
 - middleware/auth.js
- Prepared placeholder controller functions for:
 - register
 - login
 - getMe
- Registered /api/auth routes inside backend application

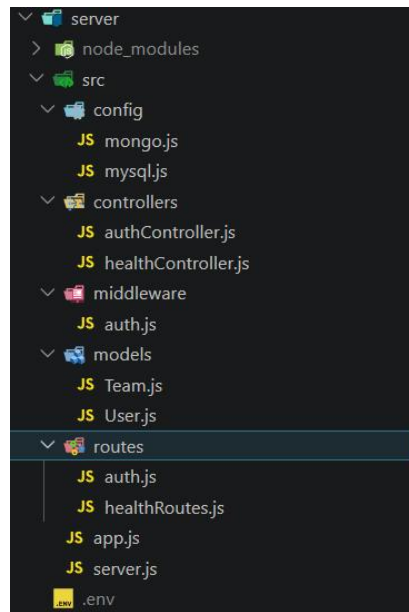


Figure 2.3 — Authentication route structure prepared inside backend architecture.

Route Verification

- Tested authentication placeholder routes using Postman
- Verified backend route registration for:
 - /api/auth/register
 - /api/auth/login

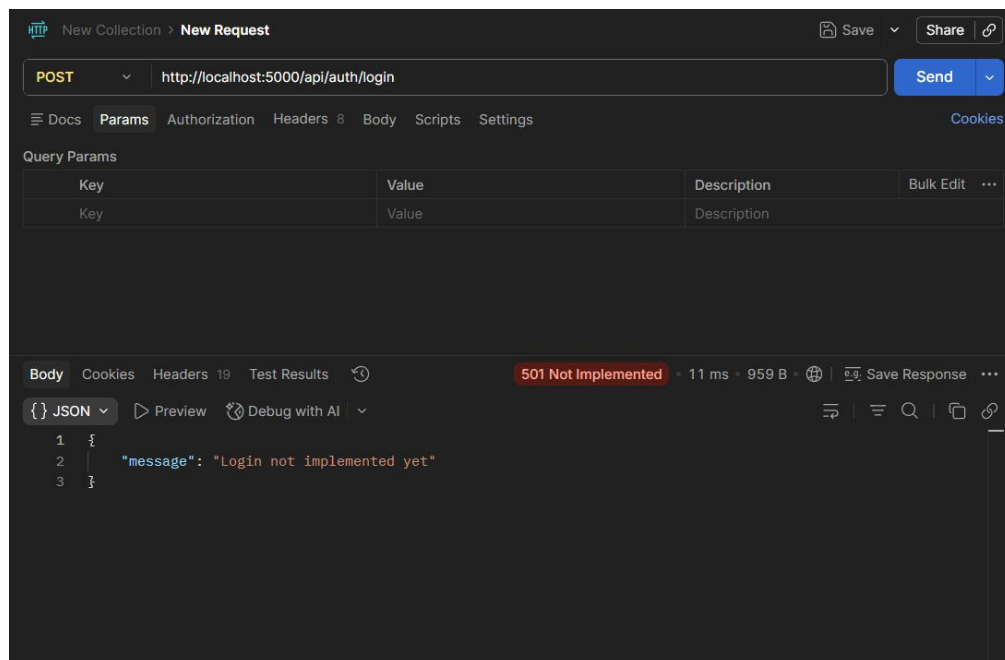


Figure 2.4 — Authentication route verification using Postman.

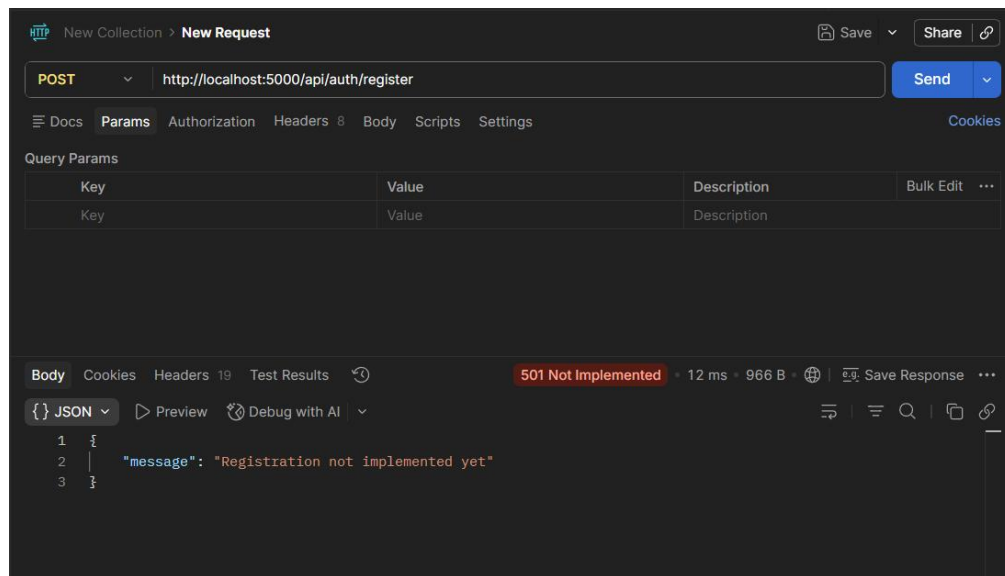


Figure 2.5 — Registration route verification using Postman.

Repository Update

- Committed schema and authentication structure changes using Git workflow
- Pushed latest backend architecture updates to GitHub repository

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
shafi@Shafin-PC MINGW64 /d/MyProject/task-management-mern (feature/project-setup)
$ git add .
warning: in the working copy of 'client/src/App.jsx', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'server/src/app.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'server/src/controllers/healthController.js', LF will be replaced by CRLF the next time Git touches it

shafi@Shafin-PC MINGW64 /d/MyProject/task-management-mern (feature/project-setup)
$ git commit -m "feat: add user and team schema architecture"
[feature/project-setup e577b3d] feat: add user and team schema architecture
9 files changed, 207 insertions(+), 17 deletions(-)
create mode 100644 server/src/controllers/authController.js
create mode 100644 server/src/middleware/auth.js
create mode 100644 server/src/models/Team.js
create mode 100644 server/src/models/User.js
create mode 100644 server/src/routes/auth.js

shafi@Shafin-PC MINGW64 /d/MyProject/task-management-mern (feature/project-setup)
$ git push
Enumerating objects: 31, done.
Counting objects: 100% (31/31), done.
Delta compression using up to 16 threads
Compressing objects: 100% (18/18), done.
Writing objects: 100% (20/20), 3.61 KiB | 370.00 KiB/s, done.
Total 20 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
To github.com:ShafinNigamana/task-management-mern.git
   fad9c1c..e577b3d  feature/project-setup -> feature/project-setup

shafi@Shafin-PC MINGW64 /d/MyProject/task-management-mern (feature/project-setup)
$

```

Figure 2.6 — Day 5 backend architecture changes pushed to GitHub.

3. Next Steps (Day-6)

- Implement signup and login functionality
- Add bcrypt password hashing
- Generate JWT tokens after login
- Implement authentication middleware protection

4. Conclusion

Day 5 focused on backend schema planning and authentication structure setup. User and Team Mongoose models were created successfully, authentication route architecture was prepared, and placeholder routes were verified using Postman.

The backend is now prepared for JWT authentication implementation in the next phase.